



Python Fundamentos

Estructuras de control

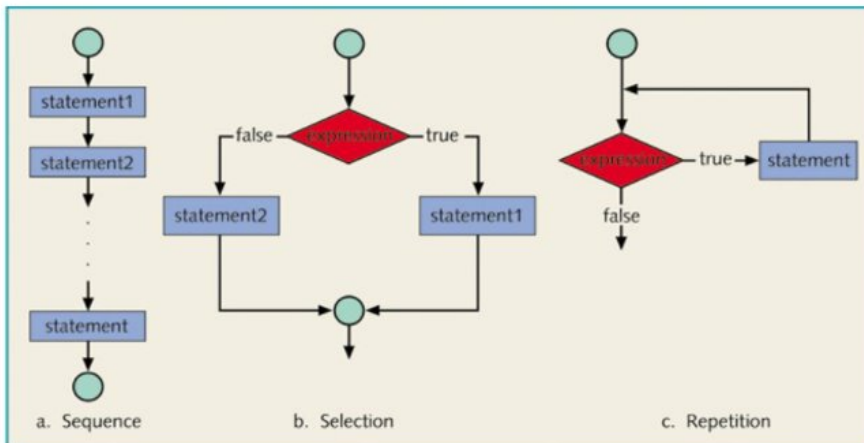
1



Las estructuras de control en Python constituyen bloques de código que permiten controlar el flujo de ejecución de un programa. Encontramos diferentes tipos de estas estructuras:

- **Condicionales:** Las estructuras condicionales permiten ejecutar bloques de código en función de que una condición se cumpla o no. Las más comunes son if, else y elif (else if).
- **Bucles:** Los bucles o ciclos permiten ejecutar un bloque de código repetidamente mientras se cumple una condición. Los bucles for y while son los más utilizados.
- **Control de Excepciones:** Las estructuras de control de excepciones (try, except, finally) permiten manejar errores y excepciones de manera predeterminada. Esta parte se estudiará específicamente en una sección específica.

Control Structures



Control de flujo:
if... else

2



La estructura if es la construcción más sencilla con la que podemos condicionar la ejecución de un bloque de código sólo si una determinada condición es verdadera.

```
python
```

```
if condicion:  
    # Bloque de código a ejecutar si la condición es verdadera
```

De este modo, en la siguiente imagen estaríamos cumpliendo con la condición impuesta por la estructura if, y se ejecutaría el código abarcado por esta estructura.

```
python
```

```
edad = 20  
  
if edad >= 18:  
    print("Eres mayor de edad.")
```



La estructura else se utiliza necesariamente junto con if para ejecutar un bloque de código cuando la condición contenida en la estructura if es falsa.

```
python

if condicion:
    # Bloque de código si la condición es verdadera
else:
    # Bloque de código si la condición es falsa
```

En este ejemplo al igual que en la anterior diapositiva, si la variable “edad” es mayor o igual a 18 se imprimirá el mensaje “Eres mayor de edad”. No obstante el valor de la variable es 15, con lo que la condición requerida por la estructura if no se cumple, y se procede a ejecutar el bloque de código contenido por else.

```
python

edad = 15

if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")
```



Es posible especificar más de una condición a comprobar si la primera de ellas no se cumple. Para ello se utiliza `elif` con el fin de evaluar múltiples condiciones en secuencia. Se verifica si la condición en el `'if'` es verdadera o falsa, y sólo en este último caso se verifican las condiciones contenidas en los bloques `'elif'` en orden de escritura.

```
python

if condicion1:
    # Bloque de código si la condición1 es verdadera
elif condicion2:
    # Bloque de código si la condicion1 es falsa y la condicion2 es verdadera
else:
    # Bloque de código si todas las condiciones anteriores son falsas
```

El código que vemos en la imagen de la derecha realiza la primera comprobación en el `'if'` inicial. Puesto que este no se cumple, pasa al primer componente `'elif'`. En esta segunda comprobación la variable `puntuación` si que es igual o superior a 70, con lo que se ejecuta el código delimitado por esta estructura. A partir de este momento no se comprueba ni ejecuta ninguna opción más. Sólo la obtención del resultado `False` en todas y cada una de las estructuras `'elif'` daría lugar a la ejecución de `'else'`.

```
python

puntuacion = 75

if puntuacion >= 90:
    print("Excelente")
elif puntuacion >= 70:
    print("Bien")
elif puntuacion >= 50:
    print("Regular")
else:
    print("Insuficiente")
```

Control de flujo:
while

3



El bucle `while` en Python se utiliza para ejecutar un bloque de código repetidamente mientras una condición específica sea verdadera. La condición se evalúa antes de ejecutar cada iteración del bucle. Si la condición es verdadera, el bloque de código dentro del bucle se ejecuta; de lo contrario, el bucle se interrumpe y la ejecución continúa con el código siguiente después del bucle.

En la imagen de la derecha vemos una variable contador que ha sido inicializada con el valor 0. Mientras esta variable sea menor que 5, el bucle `while` se ejecutará cíclicamente incrementando en 1 el valor de la variable contador. Cuando esta deje de cumplir la condición del bucle, se detendrá.

```
python

contador = 0

while contador < 5:
    print(contador)
    contador += 1
```

Es importante tener cuidado al utilizar bucles `while` para evitar bucles infinitos. Un bucle infinito ocurre cuando la condición nunca se vuelve falsa, y el código dentro del bucle se ejecuta continuamente sin detenerse.

Control de flujo:
for

4



El bucle for en Python se utiliza para iterar sobre una secuencia (como una lista, tupla, cadena o rango) o cualquier objeto iterable. Toma cada elemento del iterable, asigna el elemento a la variable especificada (variable), y luego ejecuta el bloque de código asociado. Este proceso se repite hasta que se hayan recorrido todos los elementos del iterable.

```
python

for variable in iterable:
    # Bloque de código a ejecutar en cada iteración
```

En los dos siguientes ejemplos el funcionamiento del bucle for es muy parecido. Según vemos en la imagen de la izquierda con cada iteración se está recorriendo un rango de 5 valores, que abarca las posiciones [0, 1, 2, 3, 4]. En la imagen de la derecha estamos recorriendo una lista ya predefinida con una longitud de tres elementos, en la que en cada iteración imprimirá por pantalla cada uno de los elementos en el orden en el que se han incorporado a la lista.

```
python

for i in range(5):
    print(i)
```

```
python

frutas = ["manzana", "plátano", "uva"]

for fruta in frutas:
    print(fruta)
```

Listas por
comprensión

5



La comprensión de listas en Python es una forma concisa y poderosa de crear listas. Permite definir una lista mediante una expresión más compacta en una sola línea.

- 'expresion': Una expresión que define los elementos de la nueva lista, pudiendo ser estos modificados.
- 'elemento': La variable que toma cada valor del iterable.
- 'iterable': La secuencia o iterable sobre la cual se está iterando.

```
python
```

```
nueva_lista = [expresion for elemento in iterable]
```

Viendo un ejemplo concreto, se ha construido un bucle for que itera sobre un rango de 5 valores. Tras cada iteración, toma el valor de x le aplica una elevación al cuadrado. Cuando el bucle termine, la variable cuadrados contendrá una lista cuyo contenido es [0, 1, 4, 9, 16]

```
python
```

```
cuadrados = [x**2 for x in range(5)]  
print(cuadrados)
```



Sobre las listas por comprensión en python, aparte de operar sobre cada uno de los elementos en la iteración como en el ejemplo anterior, podemos utilizar condicionales para filtrar los datos como en los siguientes ejemplos:

```
numbers = [n for n in range(11) if n % 2 == 0]
```

Salida: [0, 2, 4, 6, 8, 10]

También se puede operar a la vez que se filtran los elementos con if

```
numbers = [(n * 2) for n in range(11) if n % 2 == 0]
```

Salida: [0, 4, 8, 12, 16, 20]

```
numbers = [n for n in range(11) if n % 2 == 0 and n > 0]
```

Salida: [2, 4, 6, 8, 10]

Lambdas

6



Las lambdas en Python, también conocidas como funciones anónimas, son una característica poderosa y versátil del lenguaje.

Un lambda es una pequeña función anónima, que puede tener cualquier número de argumentos pero solo una expresión. Son útiles para crear funciones cortas y desechables sin necesidad de definirlos formalmente utilizando 'def' como veremos en el apartado de funciones.

```
python  
  
lambda argumentos: expresión
```

A continuación explicamos la sintaxis:

- 'lambda': Palabra clave para definir una función lambda
- 'argumentos': Una lista de argumentos separados por comas, similar a los argumentos de una función
- 'expresión': Una sola expresión que se evalúa y devuelve. No se permiten múltiples expresiones o declaraciones



Un ejemplo básico de una función lambda que suma dos números

```
python

suma = lambda x, y: x + y
resultado = suma(2, 3)
print(resultado) # 5
```

Como vemos en la imagen superior, definimos la función lambda que guardamos en la variable suma. Como argumentos se definen 'x' e 'y' y como expresión (lo que va a realizar la función lambda) que sume los valores de 'x' e 'y'.

Luego en la variable resultado guardamos el resultado que devuelve la función lambda pasando los valores de 2 y 3 por lo que el resultado obtenido es 5.

Este es un ejemplo básico del funcionamiento de un lambda que nos demuestra lo útil y versátil que pueden llegar a ser.



Uno de los usos más comunes de los lambdas es como argumentos para funciones de orden superior, que veremos más adelante.

- `map()`: Aplica una función a todos los elementos de un iterable

```
python

numeros = [1, 2, 3, 4]
cuadrados = map(lambda x: x**2, numeros)
print(list(cuadrados)) # [1, 4, 9, 16]
```

- `filter()`: Filtra los elementos de un iterable que cumplen con una condición

```
python

numeros = [1, 2, 3, 4]
pares = filter(lambda x: x % 2 == 0, numeros)
print(list(pares)) # [2, 4]
```

- `sorted()`: Ordena un iterable utilizando una función clave

```
estudiantes = [("Ana", 21), ("Juan", 22), ("María", 20)]
ordenado_por_edad = sorted(estudiantes, key=lambda estudiante: estudiante[1])
print(ordenado_por_edad) # [("María", 20), ("Ana", 21), ("Juan", 22)]
```



Otro de los posibles usos de los lambdas es para funciones cortas que se definen en una sola línea.

```
python
```

```
# Ejemplo de una lambda que convierte grados Celsius a Fahrenheit  
celsius_a_fahrenheit = lambda c: (c * 9/5) + 32  
print(celsius_a_fahrenheit(25)) # 77.0
```

Ventajas de los lambdas:

- Simplicidad: Los lambdas son concisos y pueden hacer el código más legible
- Función inline: Perfectos para funciones pequeñas que se utilizan solo una vez, evitando la necesidad de definir una función completa

Limitaciones:

- Inmutabilidad: Los lambdas están limitados a una sola expresión y no pueden contener múltiples declaraciones o expresiones complejas
- Legibilidad: Un uso excesivo de lambdas puede hacer que el código sea más difícil de entender, especialmente para funciones más largas o complejas

¡Gracias!